Top 50 Backend & Ledger Engineering Interview Guide

Target Role: SDE / Backend Developer (On-Campus Placements) | **Stack:** Node.js, Express, MongoDB, Auth & Distributed Ledger Architecture

1. End-to-End Workflow: How This Ledger Service Works

When a user initiates a financial transfer (e.g. transferring ■500 from User A to User B), the request follows a strict 10-step atomic workflow to ensure data integrity and auditability:

• **Step 1: Request Input & Schema Validation** — Accepts fromAccount, toAccount, amount, and idempotencyKey.

• **Step 2: Idempotency Check** — Queries database for idempotencyKey. If COMPLETED, returns cached response instantly to prevent duplicate transfers on network retries.

• **Step 3: Account Status Validation** — Ensures both sender and receiver accounts have status ACTIVE.

• **Step 4: Derived Balance Calculation** — Executes MongoDB Aggregation Pipeline (Sum(Credits) - Sum(Debits)) to verify sufficient funds.

• **Step 5: Start MongoDB Multi-Document Session** — Calls mongoose.startSession() and session.startTransaction().

• **Step 6: Create DEBIT Ledger Record** — Appends DEBIT entry for sender inside transaction session.

• **Step 7: Create CREDIT Ledger Record** — Appends CREDIT entry for receiver inside transaction session.

• **Step 8: Transition Transaction State** — Updates transaction status from PENDING to COMPLETED.

• **Step 9: Commit Transaction & Close Session** — Calls session.commitTransaction() and releases session locks.

• **Step 10: Asynchronous Notification** — Triggers Nodemailer email notification to user without blocking response.

2. Deep Concepts: What 95% of Students Fail to Understand

• **Derived Balance vs. Stored Balance:** Beginners often put a balance: Number field on the Account schema and do balance -= amount. In production finance, balance is NEVER a mutable field. It is derived on-the-fly via Sum(Credits) - Sum(Debits). This preserves a 100% verifiable audit trail.

• **Double-Entry Accounting Principle:** Money cannot be created or destroyed out of thin air. Every transaction requires equal and opposite DEBIT and CREDIT entries. The total sum across all accounts in the database must always balance to zero.

• **Idempotency Keys in Payment Systems:** If a user taps 'Pay' and their 4G connection drops before receiving a response, the mobile app retries. Without an idempotencyKey, the backend will charge the customer twice. Idempotency guarantees safe retries.

• **Multi-Document MongoDB Transactions:** Single document operations in MongoDB are atomic, but updating transactions and ledgers spans multiple collections. Multi-document ACID transactions via sessions ensure all operations either fully succeed or completely rollback.

• **Resource & Connection Cleanup:** Failing to abort transactions or close sessions inside a finally block creates database pool connection leaks that eventually crash server instances under load.

3. Edge Cases & Production Vulnerabilities

1. Race Condition / Double Spending: If User A has ■100 and sends two simultaneous ■100 requests at the exact same millisecond, both read balance as ■100 and both pass validation, leaving balance at -■100. *Fix: Use Redis distributed locks (Redlock) or optimistic concurrency control on accounts.*

2. Session Memory Leak in Try/Catch: If an error occurs during transaction execution, failing to call session.abortTransaction() and session.endSession() inside a finally block leaves active locks open in DB RAM. *Fix: Wrap session cleanup in a finally block.*

3. Floating-Point Precision Errors: In JS, 0.1 + 0.2 === 0.30000000000000004. Storing currency as decimals leads to cumulative rounding errors that break ledger balance equality. *Fix: Store all currency as Integers in lowest units (e.g., Paise or Cents).*

4. Self-Transfer Vulnerability: Passing fromAccount === toAccount creates dummy DEBIT and CREDIT entries on the same account. *Fix: Add explicit validation if (fromAccount === toAccount) throw error;.*

5. Negative / Zero / NaN Amount Injection: Passing amount: -500 actually credits the sender and debits the receiver. *Fix: Validate amount > 0 and ensure integer type.*

6. Idempotency Payload Mismatch: Reusing an idempotency key with a completely different transaction payload (e.g. ■5000 instead of ■100) returns cached response for ■100. *Fix: Hash request payload with idempotency key.*

4. Top 50 Interview Questions & Answers

Below are the top 50 technical interview questions categorized by core modules, complete with crisp, high-yield answers tailored for SDE interviews.

Category 1: Node.js Internals & Asynchronous Programming

Q1. How does Node.js handle concurrency if it is single-threaded?

Node.js runs JS code on a single thread (V8 engine), but delegates asynchronous non-blocking I/O operations (file system, network, database calls) to C++ background threads via the libuv thread pool. When an async task completes, libuv pushes its callback to the event queue to be processed by the Event Loop.

Q2. Explain the phases of the Node.js Event Loop.

1. Timers (setTimeout, setInterval)
2. Pending Callbacks (deferred I/O)
3. Idle, Prepare (internal)
4. Poll (retrieve new I/O events)
5. Check (setImmediate)
6. Close Callbacks (socket.on('close')).

Q3. What is the difference between process.nextTick(), setImmediate(), and setTimeout(fn, 0)?

process.nextTick() executes immediately after the current operation finishes (microtask queue, highest priority). setImmediate() executes in the Check phase of the Event Loop (macrotask). setTimeout(fn, 0) executes in the Timers phase after minimum delay threshold.

Q4. What is the libuv thread pool, and how do you change its size?

libuv provides OS abstraction and a pool of 4 default worker threads for heavy sync tasks (crypto, DNS, fs). You can adjust size via: process.env.UV_THREADPOOL_SIZE = 8;

Q5. Worker Threads vs Cluster Module: What is the difference?

Worker Threads (worker_threads) run multiple JS threads inside a SINGLE process sharing memory (SharedArrayBuffer), useful for CPU tasks. Cluster Module spawns multiple SEPARATE Node.js processes sharing the same port to scale across CPU cores.

Q6. What are Streams and Buffers in Node.js?

A Buffer is raw binary memory allocated outside V8 heap. A Stream reads/writes data chunk-by-chunk in real time without loading whole files into RAM (Readable, Writable, Duplex, Transform).

Q7. What is backpressure in Node.js Streams?

Backpressure happens when writable stream receives data faster than it can write. Node handles it by pausing the readable stream (stream.pause()) until writable buffer drains ('drain' event).

Q8. CommonJS (require) vs ES Modules (import): Key differences?

CommonJS is synchronous, runtime-loaded (module.exports). ES Modules are asynchronous, statically analyzed at compile time (import/export), supporting tree-shaking.

Q9. How do you prevent and detect Memory Leaks in Node.js?

Causes: global variables, uncleared setInterval, event listener leaks. Detection: Chrome DevTools (--inspect), heapdump snapshots, monitoring heap used vs heap total.

Q10. What are Unhandled Rejections and Uncaught Exceptions?

Unhandled Rejection: A Promise rejected without .catch(). Uncaught Exception: A sync error not caught in try/catch. Handle via process.on('unhandledRejection') and process.on('uncaughtException') with graceful shutdown.

Category 2: Express.js & REST API Design

Q11. How does middleware work in Express.js?

Middleware functions have access to req, res, and next. They can modify req/res, end request cycle, or call next() to pass control to the next handler in stack.

Q12. How do you handle global errors in Express.js?

Define a 4-argument middleware at the end of route stack: app.use((err, req, res, next) => { res.status(err.status || 500).json({ error: err.message }); });

Q13. What makes an HTTP method Idempotent?

An HTTP method is idempotent if executing it multiple times produces the exact same server state as executing it once. Idempotent: GET, PUT, DELETE. Non-idempotent: POST.

Q14. What are essential HTTP status codes every backend dev must know?

200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 429 Too Many Requests, 500 Internal Server Error.

Q15. What is CORS and how do you resolve CORS errors in Express?

CORS restricts cross-origin HTTP requests via browser preflight (OPTIONS). Resolved in Express using app.use(cors({ origin: 'https://frontend.com', credentials: true }));

Q16. How do you implement API Rate Limiting?

Limits requests per IP/user using Sliding Window or Token Bucket stored in Redis. Prevents DDoS/brute-force attacks. Implemented via express-rate-limit.

Q17. How do you handle Request Body Validation?

Use schema validation libraries like Zod or Joi inside middleware to validate types, sanitize inputs, and reject invalid requests with HTTP 400 before controllers run.

Q18. URI Path params vs Query params vs Request Body?

Path Params (/users/:id) identify specific resource. Query Params (/users?role=admin) filter/sort/paginate. Request Body holds complex JSON payloads for POST/PUT.

Q19. How do you implement API Versioning?

1. URI Path (Recommended): /api/v1/users. 2. Custom Header: Accept-Version: 1.0. 3. Query Param: /users?v=1.

Q20. Why put a Reverse Proxy (Nginx) in front of Node.js?

SSL/TLS termination, static asset caching/compression (Gzip), load balancing, and protecting Node.js from direct public exposure and Slowloris attacks.

Category 3: Authentication, Security & Authorization

Q21. Explain the structure of a JSON Web Token (JWT).

Consists of 3 base64url encoded parts separated by dots: Header (alg & type), Payload (claims like userId, role, exp), and Signature (HMACSHA256 of header + payload + secret).

Q22. Where to store JWTs: localStorage vs httpOnly Cookie?

localStorage is vulnerable to XSS attacks (malicious scripts steal token). httpOnly cookies cannot be accessed by JS (immune to XSS). Combine with Secure + SameSite=Strict.

Q23. What is the Access Token + Refresh Token pattern?

Access Token: Short-lived (15 min) for endpoint auth. Refresh Token: Long-lived (7 days) stored securely in DB/Redis to obtain new access tokens without re-logging in.

Q24. Why use bcrypt/Argon2 instead of SHA-256 for passwords?

SHA-256 is fast, enabling attackers to run billions of brute-force hashes/sec. bcrypt/Argon2 incorporate salts and configurable work/cost factors to slow down hashing.

Q25. Authentication vs Authorization: Key difference?

Authentication (AuthN) verifies WHO the user is (login). Authorization (AuthZ) verifies WHAT an authenticated user is permitted to do (RBAC roles).

Q26. How do you prevent NoSQL Injection in MongoDB?

Sanitize inputs using express-mongo-sanitize, validate schemas with Zod, and avoid passing raw req.body into queries (e.g. use User.find({ email: String(req.body.email) })).

Q27. What is CSRF and how do you prevent it?

CSRF tricks authenticated browsers into making unwanted cross-origin requests. Prevent via SameSite=Strict/Lax cookie attribute or anti-CSRF tokens.

Q28. What is XSS and how do you mitigate it?

XSS injects malicious JS into web pages. Mitigate via input sanitization, output escaping, Content-Security-Policy (CSP) headers, and storing tokens in httpOnly cookies.

Q29. What is Helmet.js and what headers does it set?

Express middleware setting security headers: X-Frame-Options (clickjacking), Strict-Transport-Security (HSTS), X-Content-Type-Options: nosniff, Content-Security-Policy.

Q30. Stateful (Sessions) vs Stateless (JWT) Authentication?

Stateful stores session state in DB/Redis; easy instant revocation, but requires centralized session store. Stateless (JWT) needs no DB check for verification, but revocation requires token blacklists.

Category 4: MongoDB & Database Design

Q31. SQL vs MongoDB NoSQL: When to choose MongoDB?

Choose MongoDB for dynamic/unstructured schemas, rapid iterations, hierarchical JSON objects, or high read throughput scaling via sharding. Choose SQL for complex multi-table JOINs and strict relational constraints.

Q32. Embedding vs Referencing in MongoDB?

Embedding stores sub-documents in a single doc (1-to-1 or 1-to-Few, read together). Referencing stores ObjectIds (1-to-Many or Many-to-Many) to avoid 16MB doc size limits.

Q33. What are Indexes in MongoDB and how do they help?

Indexes create B-Tree structures on fields, enabling IXSCAN instead of scanning whole collections (COLLSCAN). Speeds up reads but slightly slows down writes.

Q34. What is a Compound Index and the ESR Rule?

Compound index indexes multiple fields. ESR Rule ordering: 1. Equality fields first, 2. Sort fields second, 3. Range fields last.

Q35. What is the MongoDB Aggregation Pipeline? 5 key stages?

Processes documents through sequential transformation stages. 5 key stages: $match (filter), $group (aggregate), $project (reshape), $lookup (join), $unwind (flatten arrays).

Q36. Does MongoDB support ACID Transactions?

Yes, multi-document ACID transactions are supported on Replica Sets using mongoose.startSession() and session.startTransaction().

Q37. What are Mongoose Middleware / Hooks?

Functions executing during schema operations (pre/post save, find, validate). Used for password hashing, timestamp updates, and logging.

Q38. What is Connection Pooling in MongoDB?

Reuses a pool of open socket connections (default maxPoolSize=100) instead of opening/closing sockets per HTTP request, reducing TCP latency.

Q39. What is the 16MB document size limit in MongoDB?

Single BSON document cannot exceed 16MB. For larger data (files/logs), use GridFS (chunks files into 255KB pieces) or store files in S3 and URLs in MongoDB.

Q40. Replication vs Sharding in MongoDB?

Replication (Replica Set) copies data across nodes for High Availability & Fault Tolerance. Sharding partitions data horizontally across clusters for massive scalability.

Category 5: Ledger Systems, System Design & Architecture

Q41. What is Double-Entry Bookkeeping in Backend Ledgers?

Every transaction records equal and opposite DEBIT and CREDIT entries. Sum of debits minus credits must equal zero. Guarantees auditability and balance integrity.

Q42. How to handle Race Conditions during concurrent balance debits?

Use atomic operators ($inc with conditional query { balance: { $gte: amount } }), optimistic locking (__v), or pessimistic distributed locks via Redis (Redlock).

Q43. How to implement API Idempotency in Payment Systems?

Client sends Idempotency-Key header. Backend checks Redis/DB if key exists: if completed, returns cached result; if new, processes transaction inside DB session.

Q44. How does Redis Caching work and common eviction strategies?

Stores key-value data in RAM for sub-millisecond reads. Cache-Aside pattern reads Redis first, falls back to DB on miss. Eviction policies: LRU, LFU, TTL.

Q45. Why use Message Queues (RabbitMQ, Kafka, BullMQ)?

Decouples producers and consumers to process heavy background tasks asynchronously (emails, PDF reports, webhook retries) without blocking HTTP responses.

Q46. Monolithic vs Microservices Architecture?

Monolith: Single codebase and deployment; easy initially, hard to scale independently. Microservices: Small decoupled services communicating via REST/gRPC/PubSub; scalable but introduces distributed tracing and consistency complexity.

Q47. What is Database Connection Leak and how to avoid it?

Occurs when connections opened inside requests are not released due to unhandled errors. Avoid by using managed connection pools (Mongoose singleton) and finally blocks.

Q48. How do you implement Graceful Shutdown in Node.js?

On SIGINT/SIGTERM: Stop accepting new requests (server.close()), complete pending in-flight requests, close DB pools (mongoose.connection.close()), exit cleanly.

Q49. Why is console.log bad for production logging?

console.log is synchronous to stdout in Node (blocks event loop) and unformatted. Use structured logging (Winston, Pino) for JSON logs with timestamps and log levels.

Q50. How do you perform DB Schema Migrations with zero downtime?

Use Expand-Contract Pattern: 1. Expand (add new field without deleting old), 2. Deploy code writing to both fields, 3. Backfill old records, 4. Contract (remove old field).